



School: Campus:
Academic Year: Subject Name: Subject Code:
Semester: Program: Branch: Specialization:
Date:

Applied and Action Learning

(Learning by Doing and Discovery)

Name of the Experiment :

* Coding Phase: Pseudo Code / Flow Chart / Algorithm

Blockchain Dev Tools – Setting Up Environment:

Setting up a blockchain development environment, especially for smart contract development on platforms like Ethereum, typically involves installing several key tools and configuring them to work together.

1. Essential Software:

- Node.js and npm: Many blockchain development tools are built on Node.js and utilize npm (Node Package Manager) for installation and dependency management.

node -v

npm -v

Code Editor:

A robust code editor is crucial for writing and managing your smart contract code. Visual Studio Code (VS Code) is a popular choice due to its extensive plugin ecosystem, including Solidity extensions for syntax highlighting and linting.

Truffle or Hardhat:

These are popular development frameworks for Ethereum that provide a suite of tools for smart contract development, including compilation, testing, and deployment.

```
# Install Truffle globally
```

```
npm install -g truffle
```

```
# Verify installation
```

```
truffle version
```

- Ganache: A personal blockchain for Ethereum development that allows you to deploy contracts, develop applications, and run tests locally without interacting with a live network. You can download and install Ganache as a desktop application.

2. Additional Tools (Depending on Project Needs):

- MetaMask:

A browser extension wallet for interacting with Ethereum-based decentralized applications (dApps).

- OpenZeppelin:

Provides secure and standards-compliant smart contract templates for common functionalities.

- Ethers.js or Web3.js:

JavaScript libraries for interacting with the Ethereum blockchain from your dApp frontend.

- Infura or Alchemy:

Hosted node providers that offer convenient access to the Ethereum network without running your own full node.

- Chainlink:

For connecting smart contracts to real-world data and events through oracles.

Coding Phase: Pseudo Code / Flow Chart / Algorithm

Step

Complete Blockchain Dev Setup Demo

Step 1: Install MetaMask Wallet

Demo:

1. Go to metamask.io
2. Click "Download" and install browser extension
3. Create new wallet → Save Secret Recovery Phrase securely
4. Switch to Sepolia Testnet (we'll use this for testing)

2: Set Up Hardhat Project

```
# Create project directory
mkdir my-first-dapp
cd my-first-dapp

# Initialize npm project
npm init -y

# Install Hardhat
npm install --save-dev hardhat

# Create Hardhat project
npx hardhat
```

When prompted, choose:

- ☒ "Create a JavaScript project"
- ☒ Yes to all dependencies

Step 3: Write Your First Smart Contract

Step 4: Configure Network & Deploy

Step 5: Get Test ETH & Deploy

Demo Steps:

1. Get Sepolia ETH:
 - Go to sepoliafaucet.com
 - Connect MetaMask → Request 0.5 Sepolia ETH
2. Deploy to Testnet:
npx hardhat run scripts/deploy.js --network sepolia
3. Expected Output:

MyToken deployed to: 0x742d35Cc6634C0532925a3b8D..

Step 6: Verify on Block Explorer

Demo:

1. Copy the contract address from deployment
2. Go to sepolia.etherscan.io
3. Paste address → See your live contract!

Coding Phase: Pseudo Code / Flow Chart / Algorithm

```
PS D:\Blockchain\ALL Labs\Demo-Project> mkdir MyBlockchainProject

Directory: D:\Blockchain\ALL Labs\Demo-Project

Mode                LastWriteTime         Length Name
----                -
d-----          11/2/2025   8:42 AM                MyBlockchainProject

PS D:\Blockchain\ALL Labs\Demo-Project> cd MyBlockchainProject
```

```
PS D:\Blockchain\ALL Labs\Demo-Project\MyBlockchainProject> npm init -y
Wrote to D:\Blockchain\ALL Labs\Demo-Project\MyBlockchainProject\package.json:

{
  "name": "myblockchainproject",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
PS D:\Blockchain\ALL Labs\Demo-Project\MyBlockchainProject> npm install --save-dev hardhat @nomicfoundation/hardhat-toolbox
```

```
added 570 packages, and audited 571 packages in 2m

107 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

* Softwares used

- Solidity
- Hardhat
- Javascript
- VS Code

* Testing Phase: Compilation of Code (error detection)

```
Ⓢ PS D:\Blockchain\ALL Labs\Demo-Project\MyBlockchainProject> npx hardhat init
Error HHE3: No Hardhat config file found.

You can initialize a new project by running Hardhat with --init

For more info go to https://hardhat.org/HHE3 or run Hardhat with --show-stack-traces
```

* Implementation Phase: Final Output (no error)

DEMO-PROJECT

MyBlockchainProject

artifacts

cache

contracts

Greeter.sol

node_modules

scripts

JS deploy.js

test

JS hardhat.config.js

package-lock.json

package.json

MyBlockchainProject > contracts > Greeter.sol

```

1
2 // SPDX-License-Identifier: MIT
3 pragma solidity ^0.8.9;
4
5 contract Greeter {
6     string public greeting;
7
8     constructor(string memory _greeting) {
9         greeting = _greeting;
10    }
11
12    function setGreeting(string memory _greeting) public {
13        greeting = _greeting;
14    }
15
16    function greet() public view returns (string memory) {
17        return greeting;
18    }
19 }
20

```

MyBlockchainProject > scripts > JS deploy.js > ...

```

1 const hre = require("hardhat");
2
3 async function main() {
4     console.log("Deploying Greeter contract...");
5
6     const Greeter = await hre.ethers.getContractFactory("Greeter");
7     const greeter = await Greeter.deploy("Hello, Blockchain!");
8
9     await greeter.waitForDeployment();
10    const address = await greeter.getAddress();
11
12    console.log("✅ Greeter deployed to:", address);
13 }
14
15 main().catch((error) => {
16     console.error(error);
17     process.exitCode = 1;
18 });

```

MyBlockchainProject > JS hardhat.config.js > ...

```

1
2 require("@nomicfoundation/hardhat-toolbox");
3
4 module.exports = {
5     solidity: "0.8.19",
6     networks: {
7         localhost: {
8             url: "http://127.0.0.1:8545"
9         }
10    }
11 };

```

Implementation Phase: Final Output (no error)

```
PS D:\Blockchain\ALL Labs\Demo-Project\MyBlockchainProject> npx hardhat compile
Compiled 1 Solidity file successfully (evm target: paris).
PS D:\Blockchain\ALL Labs\Demo-Project\MyBlockchainProject> npx hardhat run scripts/deploy.js
Deploying Greeter contract...
  ✓ Greeter deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3
PS D:\Blockchain\ALL Labs\Demo-Project\MyBlockchainProject> npx hardhat test

 0 passing (1ms)

PS D:\Blockchain\ALL Labs\Demo-Project\MyBlockchainProject> |
```

```
PS D:\Blockchain\ALL Labs\Demo-Project\MyBlockchainProject> npx hardhat run scripts/deploy.js
Deploying Greeter contract...
  ✓ Greeter deployed to: 0x5FbDB2315678afecb367f032d93F642f64180aa3
PS D:\Blockchain\ALL Labs\Demo-Project\MyBlockchainProject> |
```

* Implementation Phase: Final Output (no error)

Applied and Action Learning

- Hardhat command shows available tasks
- Compilation successful
- Deployment script runs

* Observations

Hardhat Core → Toolbox Plugins → TypeScript → Node.js Modules

↓ ↓ ↓ ↓
Solidity Compiler → Ethers.js → Network Providers → Testing

1. Configuration Complexity:

- Multiple module systems caused confusion (ESM vs CommonJS)
- TypeScript integration introduced additional configuration overhead
- Version compatibility issues between Hardhat and toolbox plugins

2. Tooling Learning Curve:

- Hardhat CLI behavior varies between versions
- Initialization process inconsistent (init vs --init)
- Error messages sometimes unclear for beginners

3. Environment Setup Hurdles:

- Package management conflicts and peer dependency warnings
- File system differences between Windows PowerShell and Unix commands
- Module resolution problems requiring manual intervention

ASSESSMENT

Rubrics	Full Mark	Marks Obtained	Remarks
Concept	10		
Planning and Execution/ Practical Simulation/ Programming	10		
Result and Interpretation	10		
Record of Applied and Action Learning	10		
Viva	10		
Total	50		

Signature of the Student:

Name :

Regn. No. :

Signature of the Faculty:

Page No.....

**As applicable according to the experiment.
Two sheets per experiment (10-20) to be used.*